

32. Dessiner avec quatre nuances : lumière et ombre

Quatre nuances suffisent à faire du relief, si chacune a un rôle.

1. Ce qu'il faut comprendre

Avec si peu de nuances, les graphistes de la Game Boy donnaient **un rôle** à chacune : **0** pour le fond et les reflets, **1** et **2** pour les surfaces, **3** pour les contours et les ombres.

Le relief vient d'une convention : **la lumière arrive d'en haut à gauche**. Le bord haut et le bord gauche d'un bloc sont donc clairs, le bord bas et le bord droit sombres. L'œil comprend aussitôt : le bloc **sort** de l'écran. Inverse les bords, et le même carré devient un **trou**.

Le **tramage** fabrique une nuance qui n'existe pas : un damier de 1 et de 2, vu de loin, se lit comme une nuance intermédiaire. C'est une ruse très utilisée sur Game Boy pour les dégradés et les ombres douces.

Un conseil qui vaut pour toute la suite, **même en couleur** : un dessin doit rester lisible en quatre nuances. Si ton héros et ton décor emploient les mêmes nuances aux mêmes endroits, ils se confondent — on ne sait plus qui est où.

À toi : dessine une brique avec un joint clair (0) et une face en 2, puis un mur entier.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **580 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	49 o	← la console, à l'allumage — la mise en route de la console
main()	118 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← ecran() — attend que l'écran soit libre
AttendreAcces	12 o	← EcrireTexte — attend de pouvoir écrire une case
AdresseCase	10 o	← poser() — trouve l'adresse d'une case de l'écran
EcrireTexte	10 o	← texte() — écrit un texte, lettre par lettre
CopierTuiles	76 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran
Tuiles	224 o	← CopierTuiles — 11 lettres sur 43, et 3 tuiles dessinées
"BLOCS"	5 o	← texte() — un texte gravé
"TROUS"	5 o	← texte() — un texte gravé
"TRAME"	5 o	← texte() — un texte gravé

Lettres gravées : ABCELMORSTU

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

poser() × 3	39 o sur place, dans main()	+ AdresseCase (10 o)
texte() × 3	33 o sur place, dans main()	+ EcrireTexte (10 o)

```
ecran() × 2    21 o sur place, dans main() + AttendreVBlank (16 o)
image() × 1    3 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : trois bandes de tuiles : des blocs en relief, des trous, et une trame.
//
// L'astuce du relief : la lumière vient d'en haut à gauche.
// Bords CLAIRS en haut et à gauche + bords SOMBRES en bas et à droite → ça sort.
// L'inverse → ça s'enfonce.
// Rappel des nuances : 0 le plus clair, 1 clair, 2 sombre, 3 le plus sombre.

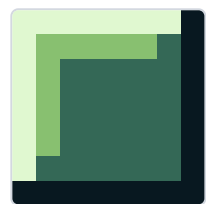
// Un bloc qui sort de l'écran : lumière en haut à gauche.
// Rangée du haut : 0 (clair) puis un 3 au bout ; la dernière rangée : tout en 3 (ombre).

#include <Tuile> // un dessin de 8 × 8 pixels
#include <ecran> // éteint ou rallume l'écran
#include <texte> // écrit un texte à l'écran
#include <poser> // pose une tuile sur une case du fond
```

2. UN DESSIN — BLOC

```
Tuile BLOC = {
    "00000003",
    "01111123",
    "01222223",
    "01222223",
    "01222223",
    "01222223",
    "01222223",
    "02222223",
    "33333333",
};

// Le même carré, bords inversés : il s'enfonce.
// Sombre en haut à gauche, clair en bas à droite.
```

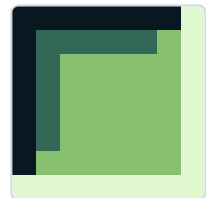


à l'écran

3. UN DESSIN — TROU

```
Tuile TROU = {
    "33333330",
    "32222210",
    "32111110",
    "32111110",
    "32111110",
    "32111110",
    "31111110",
    "00000000",
};

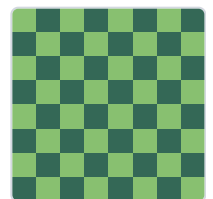
// Le tramage : un damier de 1 et de 2, lu comme une nuance entre les deux.
// De loin, l'œil mélange les pixels : on croit voir une 5e nuance.
```



à l'écran

4. UN DESSIN — TRAME

```
Tuile TRAME = {
    "12121212", "21212121", "12121212", "21212121",
    "12121212", "21212121", "12121212", "21212121",
};
```



à l'écran

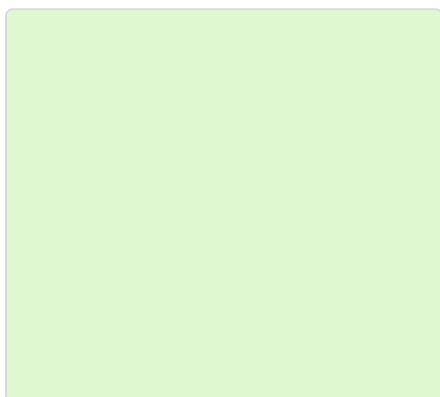
5. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                                // Le jeu commence ici.
    ecran(0);                               // Dessin écran éteint.
    texte(1, 1, "BLOCS");
    texte(1, 6, "TROUS");
    texte(1, 11, "TRAME");
    for (uint8_t c = 0; c < 20; c++) { // Les 20 colonnes :
        poser(c, 3, BLOC);               // une rangée de blocs (ligne 3),
        poser(c, 8, TROU);               // une rangée de trous (ligne 8),
        poser(c, 13, TRAME);             // une rangée de trame (ligne 13).
    }
    ecran(1);                             // On rallume.

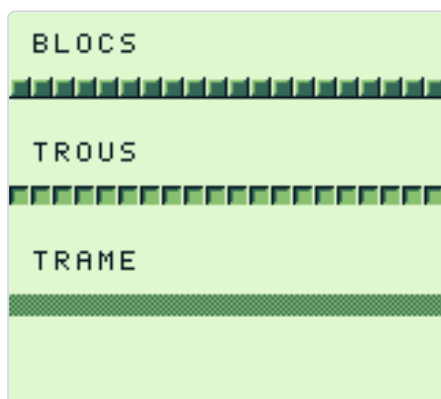
    while (true) {                         // La boucle du jeu.
        image();
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Une rangée de blocs en relief, une rangée de trous, et une bande tramée.

5. Ce que la console répond

Ces contrôles sont ceux que `verification/verifier-cours.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le bloc est clair en haut à gauche

✓ et sombre en bas à droite

✓ le trou est l'inverse

✓ la trame alterne 1 et 2

